MIA-2026-Project

March 15, 2026

1 Ventilator Pressure Prediction

USC MIA-2026, XTAI Project inspired by Kaggle Competition :

https://www.kaggle.com/competitions/ventilator-pressure-prediction

Authors :
Migule Leal fernandez (miguel.leal@rais.usc.es
Gian Paolo Bulledd (gianpaolo.bulleddu@rais.usc.es)

---

2 1. Introduction

2.1 1.1 Background

Mechanical ventilation is a critical medical intervention used when patients are unable to breathe adequately on their own. A ventilator delivers oxygen into the patient's lungs through a tube inserted into the windpipe while the patient is sedated.

Mechanical ventilation is a highly clinician-intensive process. This limitation became particularly visible during the early stages of the COVID-19 pandemic when the demand for ventilators and trained medical staff increased dramatically worldwide.

Developing and testing new ventilator control strategies is costly and complex, especially before reaching clinical trial stages. High-fidelity computational simulators provide a promising solution by enabling safe and cost-effective experimentation.

2.2 1.2 Problem Statement

Traditional ventilator control systems rely on **PID (Proportional–Integral–Derivative) controllers**.

Although PID controllers are widely used, they may struggle to adapt to patients with significantly different lung characteristics.

Existing ventilator simulators are often trained as **ensembles of models**, each corresponding to a specific lung configuration. However, lung mechanics vary continuously across patients.

Therefore, a more general **parametric modeling approach** is required.

2.3 1.3 Research Objective

The objective of this project is to develop a **data-driven simulator** capable of predicting airway pressure dynamics during mechanical ventilation.

The model must account for:

- **Control inputs** from ventilator valves
- **Lung mechanical properties**
- **Temporal airflow dynamics**

The central hypothesis is that:

> Deep learning models can better generalize across lungs with varying resistance and compliance compared to traditional PID controllers.

---

3 2. Ventilator System Overview

3.1 2.1 Experimental Setup

The dataset was generated using a **modified open-source ventilator** connected to an artificial **bellows test lung**.

The ventilator system includes:

- **Two control signals**
- **One target variable (airway pressure)**

The ventilator manipulates airflow using inspiratory and expiratory valves.

3.2 2.2 Control Inputs

3.2.1 Inspiratory Valve (`u_in`)

Continuous variable ranging from **0 to 100** representing the percentage opening of the inspiratory valve.

Value	Meaning
0	Valve closed
100	Valve fully open

3.2.2 Expiratory Valve (`u_out`)

Binary variable indicating whether the expiratory valve is open.

Value	Meaning
0	Valve closed
1	Valve open

3.3 2.3 Target Variable

3.3.1 Airway Pressure (`pressure`)

Measured airway pressure inside the respiratory circuit.

Units: **cmH O**

The goal of the model is to predict this pressure at every timestep.

---

4 3. Dataset Description

4.1 3.1 Time Series Structure

Each breathing cycle corresponds to **one time series sequence**.

Dataset characteristics:

Property	Value
Sequence length	80 timesteps
Duration	~3 seconds
Target	airway pressure
Inputs	valve controls + lung parameters

Each sequence corresponds to **one breath** identified by `breath_id`.

4.2 3.2 Dataset Columns

4.2.1 Identifiers

Feature	Description
`id`	unique identifier for each timestep
`breath_id`	identifier grouping timesteps within the same breath

4.2.2 Lung Mechanical Properties

Feature	Meaning
`R`	lung resistance
`C`	lung compliance

These parameters define the physical behavior of the lung.

Higher resistance makes airflow more difficult, while higher compliance allows the lung to expand more easily.

4.2.3 Control Signals

Feature	Meaning
`u_in`	inspiratory valve opening
`u_out`	expiratory valve state

4.2.4 Time Variable

Feature	Meaning
`time_step`	timestamp within the breath

---

5 4. Ventilator Physics

The pressure dynamics inside the lung can be approximated by the equation:

$P(t) = R \cdot F(t) + \frac{V(t)}{C}$

where:

Symbol	Meaning
(P(t))	airway pressure
(F(t))	airflow
(V(t))	lung volume
(R)	lung resistance
(C)	lung compliance

The dataset provides airflow and valve signals, and the goal is to learn a model that approximates this underlying physiological relationship.

---

6 5. Time Series Modeling

6.1 5.1 Time Series Characteristics

A time series is a sequence of observations ordered in time:

$x_1, x_2, x_3, ..., x_T$

Time series differ from traditional tabular data because observations exhibit **temporal dependencies**.

Key characteristics include:

- Autocorrelation
- Sequential structure
- Dynamic evolution

6.2 5.2 Time Series in Ventilator Data

In this dataset:

- Each breath forms a **short multivariate time series**
- Sequences are independent across breaths
- Within each breath, pressure evolves dynamically

Pressure depends on:

- Previous pressure values
- Airflow inputs
- Lung mechanical properties

Therefore, the ventilator system behaves as a **nonlinear dynamical system**.

---

7 6. Feature Engineering

Feature engineering aims to provide the model with **temporal context and physical signals** describing airflow dynamics.

The engineered features capture:

- airflow accumulation
- breathing phase transitions
- past and future airflow patterns

These signals help the model approximate the **underlying respiratory mechanics**.

(your feature engineering section can follow here)

---

8 7. Model Architecture

Sequential models are particularly suitable for this problem because they capture **temporal dependencies**.

Common architectures for this task include:

- Recurrent Neural Networks (RNN)
- Long Short-Term Memory (LSTM)
- Gated Recurrent Units (GRU)
- Transformer architectures

These models can learn nonlinear relationships between airflow control signals and airway pressure.

---

9 8. Evaluation Metric

For our model the evaluation metric is MAE. Using MAE as the training loss means that the model optimizes exactly what will be evaluated.

The **Mean Absolute Error (MAE)** measures the average absolute difference between the predicted value and the true value.

$\text{MAE} = \frac{1}{N} \sum |y_i - \hat{y}_i|$

Where:
y = true value
$\hat{y}$ = predicted value
n = number of samples

Why do we use MAE?
`Minimizing MAE makes the model estimate the conditional median of the target distribution.`

This is beneficial when: - target distribution is skewed
- there are outliers

Ventilator pressure distributions often have:
- plateaus
- jumps
- asymmetric noise

So predicting the median behavior is reasonable.

`MAE produces interpretable errors.`
Since MAE is measured in the same units as the target, the interpretation of the results is very i ntuitive for Clinicians and model evaluation.

Example:MAE = 0.5
Means on average the prediction is off by 0.5 pressure units.

`MAE penalizes errors linearly.`
Because MAE does not square the error, it is less sensitive to extreme values.

Example : True pressure value = 20

Prediction	Error	MAE	MSE
21	1	1	1
22	2	2	4
40	20	20	400

With MSE:
one bad prediction (40) would dominate the loss.
With MAE:
errors remain proportional.
This is useful when sensor noise or rare events exist.

`The gradient magnitude is constant`
Large errors do not explode gradients.
This helps when training deep sequence models such as LSTMs or Transformers.

---

10 9. Model Interpretability

To understand model behavior, we apply **SHAP (SHapley Additive Explanations)**.

SHAP assigns each feature a contribution value explaining its impact on a prediction.

A prediction can be decomposed as:

$f(x) = \phi_0 + \sum_{i=1}^{d} \phi_i$

where ($ _i$) represents the contribution of feature (i).

This allows analysis of:

- global feature importance
- local explanations for individual breaths
- temporal importance across timesteps

---

11 10. Conclusion

This project formulates ventilator simulation as a **multivariate time-series prediction problem**.

By combining:

- physics-inspired features
- sequential deep learning models
- interpretability techniques

the system aims to accurately model airway pressure dynamics across different lung configurations.

Such models may support the development of **adaptive ventilator control systems** capable of improving respiratory care.

---

12 IMPORTS

[1]:
```
import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)
import time
import warnings
import gc
import multiprocessing
import os,random, math, psutil, pickle
from datetime import datetime
import matplotlib.pyplot as plt
import seaborn as sns
from datetime import datetime

#~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
#              sklearn
#~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import LabelEncoder
from sklearn.preprocessing import RobustScaler, normalize
from sklearn.model_selection import train_test_split, GroupKFold, KFold
import tensorflow as tf

#~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
#              Keras
#~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
import tensorflow.keras.backend as K
from tensorflow.keras.losses import MeanSquaredError as mse_loss
from tensorflow.keras.utils import *
from tensorflow.keras import Sequential
from tensorflow.keras.layers import *
from tensorflow.keras.callbacks import *
from tensorflow.keras.models import Model, Sequential, model_from_json
from tensorflow.keras import regularizers

from tensorflow.keras.layers import Dense, LSTM ,Bidirectional
import tensorflow as tf
from tensorflow.keras.callbacks import ModelCheckpoint

import shap
import numpy as np
```

```
2026-03-13 13:17:47.508763: I tensorflow/core/platform/cpu_feature_guard.cc:210]
This TensorFlow binary is optimized to use available CPU instructions in
performance-critical operations.
To enable the following instructions: AVX2 FMA, in other operations, rebuild
TensorFlow with the appropriate compiler flags.
/home/gpaolo/miniconda3/envs/dl_env/lib/python3.12/site-
packages/tqdm/auto.py:21: TqdmWarning: IProgress not found. Please update
jupyter and ipywidgets. See
https://ipywidgets.readthedocs.io/en/stable/user_install.html
  from .autonotebook import tqdm as notebook_tqdm
```

13 Set Notebook Options

[2]:
```
print("Tensorflow version " + tf.__version__)
print("Num GPUs Available: ", len(tf.config.list_physical_devices('GPU')))
warnings.filterwarnings('ignore')
pd.set_option('display.max_columns', None)
pd.set_option('display.max_rows', None)
pd.set_option('display.float_format', lambda x: '%.5f' % x)
```

```
Tensorflow version 2.20.0
Num GPUs Available:  1
```

14 Functions

14.1 ADD FEATURES

This function creates engineered features for the ventilator time-series dataset, where each breath_id represents a single breathing cycle and each row corresponds to a time step within that breath.

The dataset represents ventilator breathing cycles. Each breath_id corresponds to a complete breath and contains multiple time steps describing airflow and valve behavior.

The goal of feature engineering is to provide the model with information about: - Airflow dynamics
- Accumulated lung volume
- Breathing phase transitions
- Physical lung properties

These signals help the model predict airway pressure at each time step.

Since the dataset provides airflow (u_in) and valve state (u_out), we engineer features that approximate: - Flow history - Accumulated volume - Breathing phase - Lung mechanical properties

– **Volume / Accumulation Features**
These features approximate the amount of air delivered to the lungs over time. Pressure increases as lung volume increases, so cumulative airflow helps estimate how full the lungs are during a breath. |Feature | Meaning
|———–|———————————- |`u_in` |airflow entering the lungs | |`time_step` |time |

Meaning
This approximates the integral of airflow over time.
It estimates the volume of air delivered during the breath so far.
This is important because pressure depends on lung volume.

Cumulative air-flow

Pressure in lungs increases as volume increases.
|Feature | Meaning
|———————|———————————-| |`u_in_cumsum` | Running total of airflow entering during the breath. |

Lag Features (Past Information)
Pressure depends on previous airflow because lungs have memory.
|Feature | Meaning
|———————|———————————-| |`u_in_lag1` | airflow at **previous timestep** | |`u_out_lag1` | valve state at **previous timestep**|
|`u_in_lag2` | airflow **2 timesteps ago** | |`u_in_lag3` | airflow **3 timesteps ago** | |`u_in_lag4` | airflow **4 timesteps ago** |

Forward Lag (Future Information) For models trained on entire sequences, future values help capture patterns in the breathing curve. A sudden drop in airflow indicates exhalation phase starting.

Feature	Meaning
`u_in_lag_back1`	Airflow at the **next timestep**
`u_in_lag_back2`	Airflow **two timesteps ahead**
`u_in_lag_back3`	Airflow **three timesteps ahead**
`u_in_lag_back4`	Airflow **four timesteps ahead**

Past Valve State (u_out) The variable `u_out` indicates whether the exhalation valve is open. Past valve states help detect the breathing phase transition between inhalation and exhalation. Past and future exhalation valve states.

Feature	Meaning
`u_out_lag1`	Valve state 1 timestep before
`u_out_lag2`	Valve state 2 timesteps before
`u_out_lag3`	Valve state 3 timesteps before
`u_out_lag4`	state 4 timesteps before

Future Valve State

Future valve information helps anticipate when exhalation will begin, which typically causes a rapid pressure decrease. |Feature | Meaning
|———————|———————————-| |`u_out_lag_back1` |Valve state next timestep| |`u_out_lag_back2` |Valve state two timesteps ahead| |`u_out_lag_back3` |Valve state three timesteps ahead| |`u_out_lag_back4` |Valve state four timesteps ahead|

Lung Mechanical Properties These parameters define the physical properties of the lung and strongly influence pressure behavior.

Feature	Meaning
R	Lung resistance affecting pressure from airflow
C	Lung compliance affecting pressure from lung volume
R__C	Combined lung type formed from R and C

[3]:
```
# # Functions
#from google.colab import drive
#drive.mount('/content/gdrive', force_remount=True)
#!ls "/content/gdrive/My Drive/kaggle"
def add_features(df):#022
    df['area'] = df['time_step'] * df['u_in']
    df['area'] = df.groupby('breath_id')['area'].cumsum()
    df['u_in_cumsum'] = (df['u_in']).groupby(df['breath_id']).cumsum()
    df['u_in_lag1'] = df.groupby('breath_id')['u_in'].shift(1)
    df['u_out_lag1'] = df.groupby('breath_id')['u_out'].shift(1)
    df['u_in_lag_back1'] = df.groupby('breath_id')['u_in'].shift(-1)
    df['u_out_lag_back1'] = df.groupby('breath_id')['u_out'].shift(-1)
    df['u_in_lag2'] = df.groupby('breath_id')['u_in'].shift(2)
    df['u_out_lag2'] = df.groupby('breath_id')['u_out'].shift(2)
    df['u_in_lag_back2'] = df.groupby('breath_id')['u_in'].shift(-2)
    df['u_out_lag_back2'] = df.groupby('breath_id')['u_out'].shift(-2)
    df['u_in_lag3'] = df.groupby('breath_id')['u_in'].shift(3)
    df['u_out_lag3'] = df.groupby('breath_id')['u_out'].shift(3)
    df['u_in_lag_back3'] = df.groupby('breath_id')['u_in'].shift(-3)
    df['u_out_lag_back3'] = df.groupby('breath_id')['u_out'].shift(-3)
    df['u_in_lag4'] = df.groupby('breath_id')['u_in'].shift(4)
    df['u_out_lag4'] = df.groupby('breath_id')['u_out'].shift(4)
    df['u_in_lag_back4'] = df.groupby('breath_id')['u_in'].shift(-4)
    df['u_out_lag_back4'] = df.groupby('breath_id')['u_out'].shift(-4)
    df = df.fillna(0)

    df['R'] = df['R'].astype(str)
    df['C'] = df['C'].astype(str)
    df['R__C'] = df["R"].astype(str) + '__' + df["C"].astype(str)
    df = pd.get_dummies(df)
    return df
```

15 Load data files

[4]:
```
#~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
# Load data files
#~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
path = "./data/"
```

```
start_time = time.time()
#dataset = pd.read_csv(path + 'train.csv')
dataset = pd.read_csv(path + 'train.csv.zip')
print('Load files took  %s seconds' % (time.time() - start_time))

# print datasets info
display(dataset.info())
```

```
Load files took  4.363377094268799 seconds
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6036000 entries, 0 to 6035999
Data columns (total 8 columns):
 #   Column     Dtype
---  ------     -----
 0   id         int64
 1   breath_id  int64
 2   R          int64
 3   C          int64
 4   time_step  float64
 5   u_in       float64
 6   u_out      int64
 7   pressure   float64
dtypes: float64(3), int64(5)
memory usage: 368.4 MB
```

```
None
```

16 Check Missing values

[5]:
```
# Check Missing values
dataset.isnull().sum()
```

[5]:
```
id           0
breath_id    0
R            0
C            0
time_step    0
u_in         0
u_out        0
pressure     0
dtype: int64
```

17 Data Pre-processing

[6]:
```
dataset = add_features(dataset)
print(f'Datset features {dataset.shape[1]}')
dataset.head(10)
```

```
Datset features 39
```

[6]:

	id	breath_id	time_step	u_in	u_out	pressure	area	u_in_cumsum
0	1	1	0.00000	0.08333	0	5.83749	0.00000	0.08333
1	2	1	0.03365	18.38304	0	5.90779	0.61863	18.46638
2	3	1	0.06751	22.50928	0	7.87625	2.13833	40.97565
3	4	1	0.10154	22.80882	0	11.74287	4.45439	63.78448
4	5	1	0.13576	25.35585	0	12.23499	7.89659	89.14033
5	6	1	0.16970	27.25987	0	12.86771	12.52253	116.40019
6	7	1	0.20371	27.12749	0	14.69556	18.04861	143.52768
7	8	1	0.23772	26.80773	0	15.89070	24.42142	170.33541
8	9	1	0.27178	27.86472	0	15.53919	31.99439	198.20012
9	10	1	0.30573	28.31304	0	15.75009	40.65058	226.51316

	u_in_lag1	u_out_lag1	u_in_lag_back1	u_out_lag_back1	u_in_lag2
0	0.00000	0.00000	18.38304	0.00000	0.00000
1	0.08333	0.00000	22.50928	0.00000	0.00000
2	18.38304	0.00000	22.80882	0.00000	0.08333
3	22.50928	0.00000	25.35585	0.00000	18.38304
4	22.80882	0.00000	27.25987	0.00000	22.50928
5	25.35585	0.00000	27.12749	0.00000	22.80882
6	27.25987	0.00000	26.80773	0.00000	25.35585
7	27.12749	0.00000	27.86472	0.00000	27.25987
8	26.80773	0.00000	28.31304	0.00000	27.12749
9	27.86472	0.00000	26.86676	0.00000	26.80773

	u_out_lag2	u_in_lag_back2	u_out_lag_back2	u_in_lag3	u_out_lag3
0	0.00000	22.50928	0.00000	0.00000	0.00000
1	0.00000	22.80882	0.00000	0.00000	0.00000
2	0.00000	25.35585	0.00000	0.00000	0.00000
3	0.00000	27.25987	0.00000	0.08333	0.00000
4	0.00000	27.12749	0.00000	18.38304	0.00000
5	0.00000	26.80773	0.00000	22.50928	0.00000
6	0.00000	27.86472	0.00000	22.80882	0.00000
7	0.00000	28.31304	0.00000	25.35585	0.00000
8	0.00000	26.86676	0.00000	27.25987	0.00000
9	0.00000	26.76280	0.00000	27.12749	0.00000

	u_in_lag_back3	u_out_lag_back3	u_in_lag4	u_out_lag4	u_in_lag_back4
0	22.80882	0.00000	0.00000	0.00000	25.35585
1	25.35585	0.00000	0.00000	0.00000	27.25987

```
2        27.25987          0.00000    0.00000     0.00000        27.12749
3        27.12749          0.00000    0.00000     0.00000        26.80773
4        26.80773          0.00000    0.08333     0.00000        27.86472
5        27.86472          0.00000   18.38304     0.00000        28.31304
6        28.31304          0.00000   22.50928     0.00000        26.86676
7        26.86676          0.00000   22.80882     0.00000        26.76280
8        26.76280          0.00000   25.35585     0.00000        27.99327
9        27.99327          0.00000   27.25987     0.00000        26.78990

   u_out_lag_back4  R_20    R_5   R_50   C_10   C_20  C_50  R__C_20__10  \
0          0.00000  True  False  False  False  False  True         False
1          0.00000  True  False  False  False  False  True         False
2          0.00000  True  False  False  False  False  True         False
3          0.00000  True  False  False  False  False  True         False
4          0.00000  True  False  False  False  False  True         False
5          0.00000  True  False  False  False  False  True         False
6          0.00000  True  False  False  False  False  True         False
7          0.00000  True  False  False  False  False  True         False
8          0.00000  True  False  False  False  False  True         False
9          0.00000  True  False  False  False  False  True         False

   R__C_20__20  R__C_20__50  R__C_50__10  R__C_50__20  R__C_50__50  \
0        False         True        False        False        False
1        False         True        False        False        False
2        False         True        False        False        False
3        False         True        False        False        False
4        False         True        False        False        False
5        False         True        False        False        False
6        False         True        False        False        False
7        False         True        False        False        False
8        False         True        False        False        False
9        False         True        False        False        False

   R__C_5__10  R__C_5__20  R__C_5__50
0       False       False       False
1       False       False       False
2       False       False       False
3       False       False       False
4       False       False       False
5       False       False       False
6       False       False       False
7       False       False       False
8       False       False       False
9       False       False       False
```

18 TARGET EXTRACTION

[7]:
```
# ------------------------------------------------
# Reshape target to 3D array for LSTM
# ------------------------------------------------
# Ensure sorted
dataset = dataset.sort_values(["breath_id", "time_step"])
targets = dataset['pressure'].values.reshape(-1, 80)
n = targets.shape[0]
```

19 Dataset Split train,val,test

[8]:
```
# Correct Sequence-Preserving Split
# train 50% of the data, validate 10%, and test on the remaining 40%
# Entire breaths stay together
# No timestep from one breath appears in multiple splits
# No leakage across sequences
# Proper LSTM training


# Get unique sequences
breaths = dataset["breath_id"].unique()
n = len(breaths)

train_end = int(0.5 * n)
val_end   = int(0.6 * n)

train_ids = breaths[:train_end]
val_ids   = breaths[train_end:val_end]
test_ids  = breaths[val_end:]

train = dataset[dataset["breath_id"].isin(train_ids)].copy()
val   = dataset[dataset["breath_id"].isin(val_ids)].copy()
test  = dataset[dataset["breath_id"].isin(test_ids)].copy()

print(train["breath_id"].nunique(),
      val["breath_id"].nunique(),
      test["breath_id"].nunique())

# Extra Safety Check
assert set(train["breath_id"]).isdisjoint(val["breath_id"])
assert set(train["breath_id"]).isdisjoint(test["breath_id"])
assert set(val["breath_id"]).isdisjoint(test["breath_id"])

print("No sequence leakage ")
```

```
print("Train shape:", train.shape)
print("Val shape:", val.shape)
print("Test shape:", test.shape)
```

```
37725 7545 30180
No sequence leakage
Train shape: (3018000, 39)
Val shape: (603600, 39)
Test shape: (2414400, 39)
```

20 Make targets

[9]:
```
y_train = train['pressure'].values.reshape(-1,80)
y_val   = val['pressure'].values.reshape(-1,80)
y_test  = test['pressure'].values.reshape(-1,80)

print("Train Targets shape:", y_train.shape)
print("Val Targets shape:", y_val.shape)
print("Test Targets shape:", y_test.shape)

#================================
# Free memory
#================================
del targets
gc.collect()
```

```
Train Targets shape: (37725, 80)
Val Targets shape: (7545, 80)
Test Targets shape: (30180, 80)
```

[9]: 20

21 Filter columns for training

[10]:
```
#================================
# Filter columns for training
#================================

drop_cols = ['pressure','id','breath_id','time_step']

train.drop(drop_cols, axis=1, inplace=True)
val.drop(drop_cols, axis=1, inplace=True)
test.drop(drop_cols, axis=1, inplace=True)

features = list(train.columns)
print('Number of feature columns =', len(features))
```

```
#================================
# Free memory
#================================
del dataset
gc.collect()
```

```
Number of feature columns = 35
```

[10]: 0

21.1 SCALING

[11]:
```
# ------------------------------------------------
# SCALING
# ------------------------------------------------
RS = RobustScaler()
train = RS.fit_transform(train)
test  = RS.transform(test)
val   = RS.transform(val)
```

21.2 RESHAPE train,val,test to sequences

[12]:
```
# reshape train and test to 3D array for LSTM
# ------------------------------------------------
# RESHAPE TO SEQUENCES
# ------------------------------------------------
train = train.reshape(-1, 80, train.shape[-1])
val   = val.reshape(-1, 80, val.shape[-1])
test  = test.reshape(-1, 80, test.shape[-1])

print("Train shape:", train.shape)
print("Val shape:", val.shape)
print("Test shape:", test.shape)
```

```
Train shape: (37725, 80, 35)
Val shape: (7545, 80, 35)
Test shape: (30180, 80, 35)
```

[13]:
```
print("Train Targets shape:", y_train.shape)
print("Val Targets shape:", y_val.shape)
print("Test Targets shape:", y_test.shape)
```

```
Train Targets shape: (37725, 80)
Val Targets shape: (7545, 80)
Test Targets shape: (30180, 80)
```

22 MODEL

[14]:
```
# ------------------------------------------------
# MODEL
# ------------------------------------------------
input_shape = train.shape[2]
model = Sequential()

model.add(Bidirectional(
    LSTM(80, return_sequences=True),
    input_shape=(80, input_shape)
))

model.add(Bidirectional(LSTM(80, return_sequences=True)))
model.add(Bidirectional(LSTM(80, return_sequences=True)))

model.add(TimeDistributed(Dense(80)))
model.add(LeakyReLU(negative_slope=0.1))

model.add(TimeDistributed(Dense(80)))
model.add(LeakyReLU(negative_slope=0.1))

model.add(TimeDistributed(Dense(1)))

model.add(Flatten())

# ------------------------------------------------
# COMPILE nd set optimizer
# ------------------------------------------------
batch_size = 128
epochs = 500
steps_per_epoch = len(train) // batch_size
total_steps = steps_per_epoch * epochs

lr_schedule = tf.keras.optimizers.schedules.CosineDecay(
    initial_learning_rate=3e-4,
    decay_steps=total_steps,
    alpha=0.01
)

optimizer = tf.keras.optimizers.AdamW(
    learning_rate=lr_schedule,
    weight_decay=1e-4
)

model.compile(
    optimizer=optimizer,
```

```
    loss='mae',
    metrics=['mae']
)

model.summary()

# ------------------------------------------------
# CALLBACKS
# ------------------------------------------------
early_stop = EarlyStopping(
    monitor='val_loss',
    patience=5,
    restore_best_weights=True
)

# ------------------------------------------------
# Train
# ------------------------------------------------
history = model.fit(
    train,
    y_train,
    validation_data=(val, y_val),
    epochs=epochs,
    batch_size=batch_size,
    callbacks=[early_stop],
    verbose=1
)

# ------------------------------------------------
# Save the model
# ------------------------------------------------
model.save('/ventilator.keras')
```

```
WARNING: All log messages before absl::InitializeLog() is called are written to
STDERR
I0000 00:00:1773404300.820110   39761 gpu_device.cc:2020] Created device
/job:localhost/replica:0/task:0/device:GPU:0 with 5580 MB memory:  -> device: 0,
name: NVIDIA GeForce RTX 3070 Laptop GPU, pci bus id: 0000:01:00.0, compute
capability: 8.6
```

Model: "sequential"

Layer (type)	Output Shape	Param #
bidirectional (Bidirectional)	(None, 80, 160)	74,240

```
 bidirectional_1 (Bidirectional)   (None, 80, 160)              154,240

 bidirectional_2 (Bidirectional)   (None, 80, 160)              154,240

 time_distributed                  (None, 80, 80)                12,880
 (TimeDistributed)

 leaky_re_lu (LeakyReLU)           (None, 80, 80)                    0

 time_distributed_1                (None, 80, 80)                6,480
 (TimeDistributed)

 leaky_re_lu_1 (LeakyReLU)         (None, 80, 80)                    0

 time_distributed_2                (None, 80, 1)                    81
 (TimeDistributed)

 flatten (Flatten)                 (None, 80)                        0


 Total params: 402,161 (1.53 MB)

 Trainable params: 402,161 (1.53 MB)

 Non-trainable params: 0 (0.00 B)

Epoch 1/500

2026-03-13 13:18:32.750494: I
external/local_xla/xla/stream_executor/cuda/cuda_dnn.cc:473] Loaded cuDNN
version 91900

295/295             41s 105ms/step -
loss: 2.5733 - mae: 2.5733 - val_loss: 1.0394 - val_mae: 1.0394
Epoch 2/500
295/295             28s 96ms/step -
loss: 0.9618 - mae: 0.9618 - val_loss: 0.8810 - val_mae: 0.8810
Epoch 3/500
295/295             28s 95ms/step -
loss: 0.8261 - mae: 0.8261 - val_loss: 0.7489 - val_mae: 0.7489
Epoch 4/500
295/295             28s 96ms/step -
loss: 0.7349 - mae: 0.7349 - val_loss: 0.6978 - val_mae: 0.6978
Epoch 5/500
295/295             30s 100ms/step -
loss: 0.6929 - mae: 0.6929 - val_loss: 0.6583 - val_mae: 0.6583
Epoch 6/500
```

```
295/295            30s 103ms/step -
loss: 0.6534 - mae: 0.6534 - val_loss: 0.6878 - val_mae: 0.6878
Epoch 7/500
295/295            29s 100ms/step -
loss: 0.6333 - mae: 0.6333 - val_loss: 0.6173 - val_mae: 0.6173
Epoch 8/500
295/295            31s 106ms/step -
loss: 0.6060 - mae: 0.6060 - val_loss: 0.6077 - val_mae: 0.6077
Epoch 9/500
295/295            31s 103ms/step -
loss: 0.5880 - mae: 0.5880 - val_loss: 0.5659 - val_mae: 0.5659
Epoch 10/500
295/295            31s 106ms/step -
loss: 0.5655 - mae: 0.5655 - val_loss: 0.5662 - val_mae: 0.5662
Epoch 11/500
295/295            31s 103ms/step -
loss: 0.5590 - mae: 0.5590 - val_loss: 0.5331 - val_mae: 0.5331
Epoch 12/500
295/295            31s 107ms/step -
loss: 0.5411 - mae: 0.5411 - val_loss: 0.5328 - val_mae: 0.5328
Epoch 13/500
295/295            32s 110ms/step -
loss: 0.5295 - mae: 0.5295 - val_loss: 0.5420 - val_mae: 0.5420
Epoch 14/500
295/295            32s 108ms/step -
loss: 0.5191 - mae: 0.5191 - val_loss: 0.5055 - val_mae: 0.5055
Epoch 15/500
295/295            32s 110ms/step -
loss: 0.5136 - mae: 0.5136 - val_loss: 0.5062 - val_mae: 0.5062
Epoch 16/500
295/295            33s 112ms/step -
loss: 0.5327 - mae: 0.5327 - val_loss: 0.5057 - val_mae: 0.5057
Epoch 17/500
295/295            35s 117ms/step -
loss: 0.5058 - mae: 0.5058 - val_loss: 0.5025 - val_mae: 0.5025
Epoch 18/500
295/295            33s 110ms/step -
loss: 0.4986 - mae: 0.4986 - val_loss: 0.4908 - val_mae: 0.4908
Epoch 19/500
295/295            32s 110ms/step -
loss: 0.4941 - mae: 0.4941 - val_loss: 0.5094 - val_mae: 0.5094
Epoch 20/500
295/295            32s 109ms/step -
loss: 0.4799 - mae: 0.4799 - val_loss: 0.4846 - val_mae: 0.4846
Epoch 21/500
295/295            31s 106ms/step -
loss: 0.4958 - mae: 0.4958 - val_loss: 0.4869 - val_mae: 0.4869
Epoch 22/500
```

```
295/295            33s 112ms/step -
loss: 0.4845 - mae: 0.4845 - val_loss: 0.4839 - val_mae: 0.4839
Epoch 23/500
295/295            33s 112ms/step -
loss: 0.4819 - mae: 0.4819 - val_loss: 0.4715 - val_mae: 0.4715
Epoch 24/500
295/295            33s 113ms/step -
loss: 0.4755 - mae: 0.4755 - val_loss: 0.4691 - val_mae: 0.4691
Epoch 25/500
295/295            33s 113ms/step -
loss: 0.4692 - mae: 0.4692 - val_loss: 0.4795 - val_mae: 0.4795
Epoch 26/500
295/295            35s 119ms/step -
loss: 0.4638 - mae: 0.4638 - val_loss: 0.4697 - val_mae: 0.4697
Epoch 27/500
295/295            34s 117ms/step -
loss: 0.4600 - mae: 0.4600 - val_loss: 0.4515 - val_mae: 0.4515
Epoch 28/500
295/295            33s 110ms/step -
loss: 0.4536 - mae: 0.4536 - val_loss: 0.4540 - val_mae: 0.4540
Epoch 29/500
295/295            33s 112ms/step -
loss: 0.4556 - mae: 0.4556 - val_loss: 0.4636 - val_mae: 0.4636
Epoch 30/500
295/295            32s 109ms/step -
loss: 0.4680 - mae: 0.4680 - val_loss: 0.4816 - val_mae: 0.4816
Epoch 31/500
295/295            33s 112ms/step -
loss: 0.4610 - mae: 0.4610 - val_loss: 0.4498 - val_mae: 0.4498
Epoch 32/500
295/295            33s 110ms/step -
loss: 0.4506 - mae: 0.4506 - val_loss: 0.4550 - val_mae: 0.4550
Epoch 33/500
295/295            34s 116ms/step -
loss: 0.4514 - mae: 0.4514 - val_loss: 0.4499 - val_mae: 0.4499
Epoch 34/500
295/295            33s 113ms/step -
loss: 0.4452 - mae: 0.4452 - val_loss: 0.4473 - val_mae: 0.4473
Epoch 35/500
295/295            34s 114ms/step -
loss: 0.4361 - mae: 0.4361 - val_loss: 0.4526 - val_mae: 0.4526
Epoch 36/500
295/295            33s 113ms/step -
loss: 0.4394 - mae: 0.4394 - val_loss: 0.4543 - val_mae: 0.4543
Epoch 37/500
295/295            33s 113ms/step -
loss: 0.4349 - mae: 0.4349 - val_loss: 0.4321 - val_mae: 0.4321
Epoch 38/500
```

```
295/295             33s 111ms/step -
loss: 0.4314 - mae: 0.4314 - val_loss: 0.4232 - val_mae: 0.4232
Epoch 39/500
295/295             33s 113ms/step -
loss: 0.4310 - mae: 0.4310 - val_loss: 0.4437 - val_mae: 0.4437
Epoch 40/500
295/295             33s 113ms/step -
loss: 0.4300 - mae: 0.4300 - val_loss: 0.4343 - val_mae: 0.4343
Epoch 41/500
295/295             34s 114ms/step -
loss: 0.4252 - mae: 0.4252 - val_loss: 0.4230 - val_mae: 0.4230
Epoch 42/500
295/295             33s 112ms/step -
loss: 0.4204 - mae: 0.4204 - val_loss: 0.4367 - val_mae: 0.4367
Epoch 43/500
295/295             34s 116ms/step -
loss: 0.4264 - mae: 0.4264 - val_loss: 0.4547 - val_mae: 0.4547
Epoch 44/500
295/295             33s 112ms/step -
loss: 0.4191 - mae: 0.4191 - val_loss: 0.4209 - val_mae: 0.4209
Epoch 45/500
295/295             34s 114ms/step -
loss: 0.4202 - mae: 0.4202 - val_loss: 0.4222 - val_mae: 0.4222
Epoch 46/500
295/295             34s 116ms/step -
loss: 0.4088 - mae: 0.4088 - val_loss: 0.4266 - val_mae: 0.4266
Epoch 47/500
295/295             34s 114ms/step -
loss: 0.4170 - mae: 0.4170 - val_loss: 0.4414 - val_mae: 0.4414
Epoch 48/500
295/295             34s 115ms/step -
loss: 0.4232 - mae: 0.4232 - val_loss: 0.4267 - val_mae: 0.4267
Epoch 49/500
295/295             34s 116ms/step -
loss: 0.4145 - mae: 0.4145 - val_loss: 0.4289 - val_mae: 0.4289
```

23 EVALUATION

[15]:
```python
# ------------------------------------------------
# EVALUATION
# ------------------------------------------------
val_loss, val_mae = model.evaluate(val, y_val, verbose=0)
print(f"\nValidation MAE: {val_mae:.5f}")

test_loss, test_mae = model.evaluate(test, y_test, verbose=0)
print(f"\nTest MAE: {test_mae:.5f}")
```

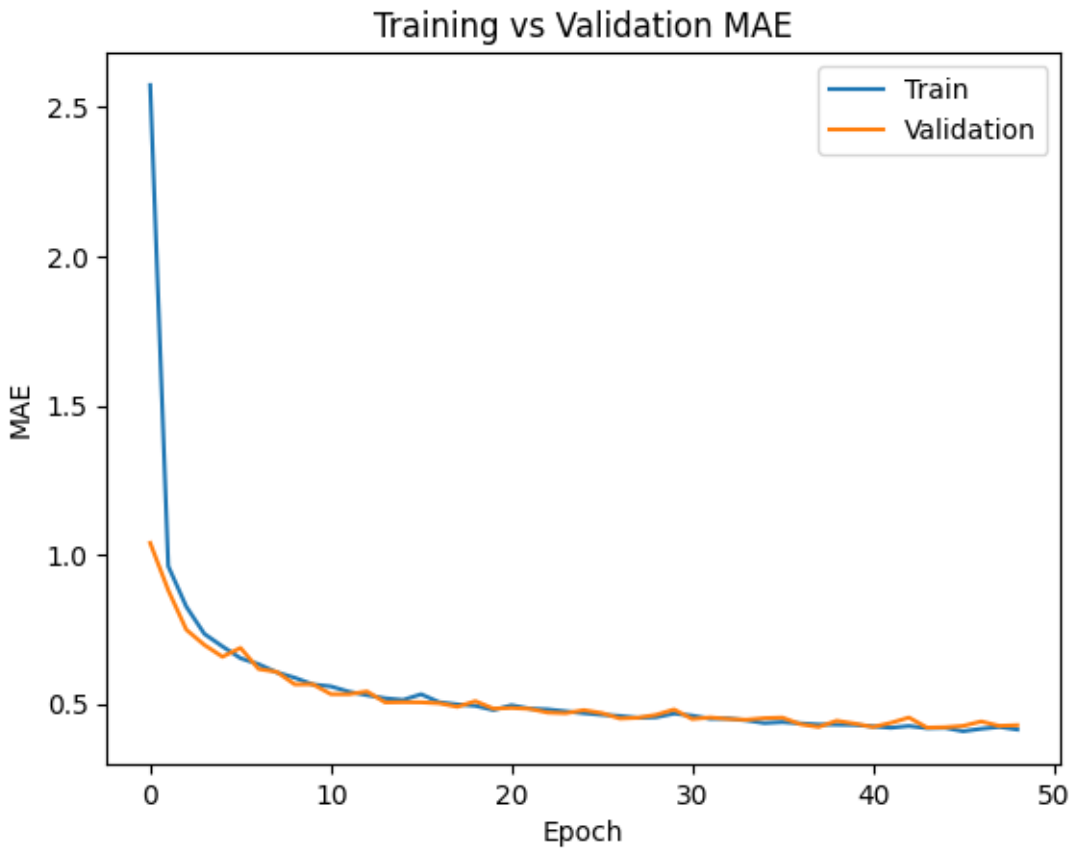
```
Validation MAE: 0.42092

Test MAE: 0.42103
```

24 PLOTS

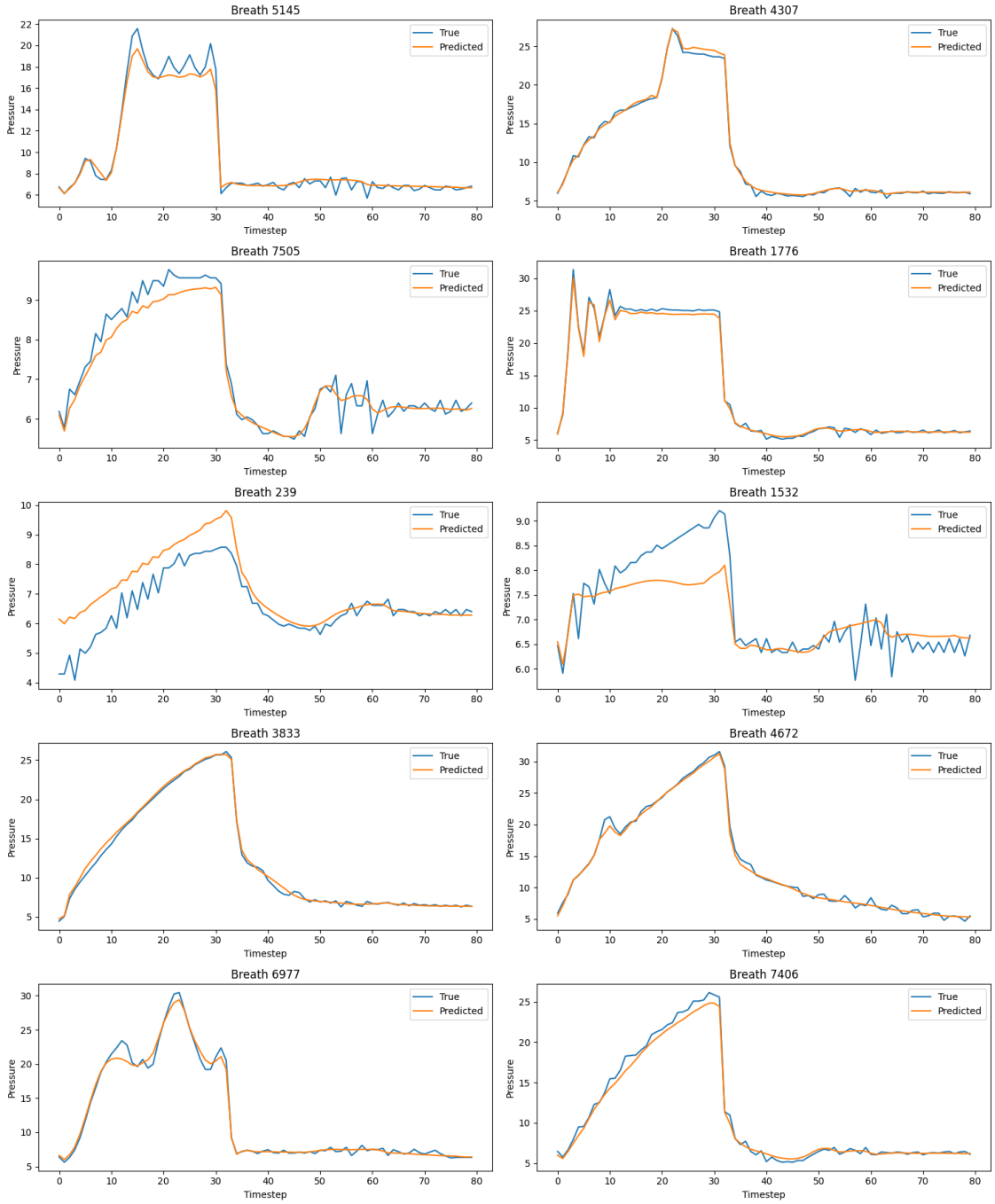
[16]:
```python
# ----------------------------------------------------
# PLOT TRAINING HISTORY
# ----------------------------------------------------
print('='*25,'PLOT TRAINING HISTORY','='*25)
plt.figure()
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title("Training vs Validation MAE")
plt.xlabel("Epoch")
plt.ylabel("MAE")
plt.legend(["Train", "Validation"])
plt.show()

# ----------------------------------------------------
# PREDICTIONS on validation set
# ----------------------------------------------------
y_pred = model.predict(val)
# ----------------------------------------------------
# Plots of 10 single breath
# ----------------------------------------------------
print('='*25,'PLOT BREATH SAMPLES ON VALIDATION','='*25)
num_samples = 10
indices = np.random.choice(len(y_val), num_samples, replace=False)

plt.figure(figsize=(15, 18))

for i, idx in enumerate(indices):
    plt.subplot(5, 2, i + 1)
    plt.plot(y_val[idx].flatten())
    plt.plot(y_pred[idx].flatten())
    plt.title(f"Breath {idx}")
    plt.xlabel("Timestep")
    plt.ylabel("Pressure")
    plt.legend(["True", "Predicted"])

plt.tight_layout()
plt.show()


# PREDICTIONS on validation set
```

```
# ------------------------------------------------
print('='*25,'PLOT BREATH SAMPLES ON TEST','='*25)
y_pred = model.predict(test)
# ------------------------------------------------
# Plots of 10 single breath
# ------------------------------------------------
num_samples = 10
indices = np.random.choice(len(y_test), num_samples, replace=False)

plt.figure(figsize=(15, 18))

for i, idx in enumerate(indices):
    plt.subplot(5, 2, i + 1)
    plt.plot(y_test[idx].flatten())
    plt.plot(y_pred[idx].flatten())
    plt.title(f"Breath {idx}")
    plt.xlabel("Timestep")
    plt.ylabel("Pressure")
    plt.legend(["True", "Predicted"])
plt.tight_layout()
plt.show()
```

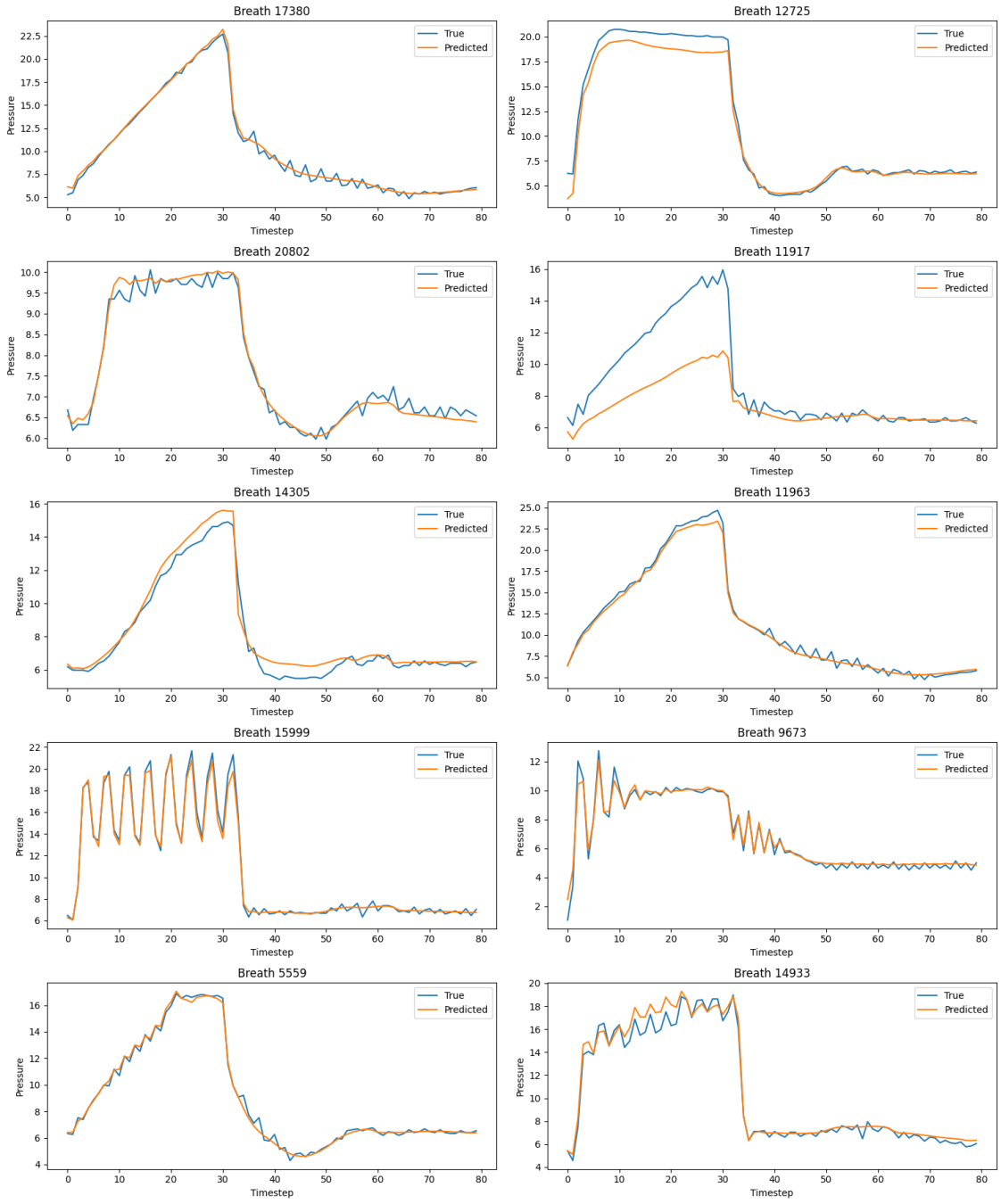
```
========================= PLOT TRAINING HISTORY =========================
```




```
236/236             10s 35ms/step
========================= PLOT BREATH SAMPLES ON VALIDATION
=========================
```




```
========================= PLOT BREATH SAMPLES ON TEST =========================
944/944                   28s 30ms/step
```



25 Retrain on all train set

- We have tor ebuild the model and train with whole train dataset to a number of epochs close to the early stopping value

[17]:
```
# ------------------------------------------------
# MODEL
# ------------------------------------------------
input_shape = train.shape[2]
model = Sequential()

model.add(Bidirectional(
    LSTM(80, return_sequences=True),
    input_shape=(80, input_shape)
))

model.add(Bidirectional(LSTM(80, return_sequences=True)))
model.add(Bidirectional(LSTM(80, return_sequences=True)))

model.add(TimeDistributed(Dense(80)))
model.add(LeakyReLU(negative_slope=0.1))

model.add(TimeDistributed(Dense(80)))
model.add(LeakyReLU(negative_slope=0.1))

model.add(TimeDistributed(Dense(1)))

model.add(Flatten())

# ------------------------------------------------
# COMPILE nd set optimizer
# ------------------------------------------------
batch_size = 128
epochs = 80
steps_per_epoch = len(train) // batch_size
total_steps = steps_per_epoch * epochs

lr_schedule = tf.keras.optimizers.schedules.CosineDecay(
    initial_learning_rate=3e-4,
    decay_steps=total_steps,
    alpha=0.01
)

optimizer = tf.keras.optimizers.AdamW(
    learning_rate=lr_schedule,
    weight_decay=1e-4
)

model.compile(
    optimizer=optimizer,
    loss='mae',
    metrics=['mae']
```

```
)

model.summary()

# ------------------------------------------------
# CALLBACKS
# ------------------------------------------------
early_stop = EarlyStopping(
    monitor='val_loss',
    patience=5,
    restore_best_weights=True
)

# ------------------------------------------------
# Train
# ------------------------------------------------
# Make new train set
X_full = np.concatenate([train, val], axis=0)
y_full = np.concatenate([y_train, y_val], axis=0)

history = model.fit(
    X_full,
    y_full,
    epochs=epochs,
    batch_size=batch_size,
    verbose=1
)

# ------------------------------------------------
# Save the model
# ------------------------------------------------
model.save('/ventilator_retrained.keras')
```

```
Model: "sequential_1"
```

Layer (type)	Output Shape	Param #
bidirectional_3 (Bidirectional)	(None, 80, 160)	74,240
bidirectional_4 (Bidirectional)	(None, 80, 160)	154,240
bidirectional_5 (Bidirectional)	(None, 80, 160)	154,240
time_distributed_3 (TimeDistributed)	(None, 80, 80)	12,880

```
 leaky_re_lu_2 (LeakyReLU)        (None, 80, 80)                  0

 time_distributed_4               (None, 80, 80)              6,480
 (TimeDistributed)

 leaky_re_lu_3 (LeakyReLU)        (None, 80, 80)                  0

 time_distributed_5               (None, 80, 1)                  81
 (TimeDistributed)

 flatten_1 (Flatten)              (None, 80)                      0


 Total params: 402,161 (1.53 MB)

 Trainable params: 402,161 (1.53 MB)

 Non-trainable params: 0 (0.00 B)

Epoch 1/80
354/354             46s 101ms/step -
loss: 2.1499 - mae: 2.1499
Epoch 2/80
354/354             37s 103ms/step -
loss: 0.8932 - mae: 0.8932
Epoch 3/80
354/354             36s 102ms/step -
loss: 0.7244 - mae: 0.7244
Epoch 4/80
354/354             37s 103ms/step -
loss: 0.6743 - mae: 0.6743
Epoch 5/80
354/354             37s 105ms/step -
loss: 0.6287 - mae: 0.6287
Epoch 6/80
354/354             38s 107ms/step -
loss: 0.6017 - mae: 0.6017
Epoch 7/80
354/354             38s 107ms/step -
loss: 0.5741 - mae: 0.5741
Epoch 8/80
354/354             38s 106ms/step -
loss: 0.5555 - mae: 0.5555
Epoch 9/80
354/354             39s 110ms/step -
loss: 0.5466 - mae: 0.5466
```

```
Epoch 10/80
354/354             38s 108ms/step -
loss: 0.5284 - mae: 0.5284
Epoch 11/80
354/354             38s 107ms/step -
loss: 0.5145 - mae: 0.5145
Epoch 12/80
354/354             39s 109ms/step -
loss: 0.5076 - mae: 0.5076
Epoch 13/80
354/354             38s 107ms/step -
loss: 0.5060 - mae: 0.5060
Epoch 14/80
354/354             39s 109ms/step -
loss: 0.5023 - mae: 0.5023
Epoch 15/80
354/354             39s 110ms/step -
loss: 0.4841 - mae: 0.4841
Epoch 16/80
354/354             39s 109ms/step -
loss: 0.4804 - mae: 0.4804
Epoch 17/80
354/354             39s 109ms/step -
loss: 0.4725 - mae: 0.4725
Epoch 18/80
354/354             39s 110ms/step -
loss: 0.4685 - mae: 0.4685
Epoch 19/80
354/354             36s 103ms/step -
loss: 0.4661 - mae: 0.4661
Epoch 20/80
354/354             36s 103ms/step -
loss: 0.4619 - mae: 0.4619
Epoch 21/80
354/354             36s 101ms/step -
loss: 0.4583 - mae: 0.4583
Epoch 22/80
354/354             37s 104ms/step -
loss: 0.4526 - mae: 0.4526
Epoch 23/80
354/354             36s 102ms/step -
loss: 0.4504 - mae: 0.4504
Epoch 24/80
354/354             37s 104ms/step -
loss: 0.4482 - mae: 0.4482
Epoch 25/80
354/354             36s 103ms/step -
loss: 0.4436 - mae: 0.4436
```

```
Epoch 26/80
354/354             36s 103ms/step -
loss: 0.4403 - mae: 0.4403
Epoch 27/80
354/354             37s 103ms/step -
loss: 0.4364 - mae: 0.4364
Epoch 28/80
354/354             35s 100ms/step -
loss: 0.4325 - mae: 0.4325
Epoch 29/80
354/354             36s 103ms/step -
loss: 0.4289 - mae: 0.4289
Epoch 30/80
354/354             38s 106ms/step -
loss: 0.4255 - mae: 0.4255
Epoch 31/80
354/354             36s 102ms/step -
loss: 0.4252 - mae: 0.4252
Epoch 32/80
354/354             37s 104ms/step -
loss: 0.4216 - mae: 0.4216
Epoch 33/80
354/354             36s 103ms/step -
loss: 0.4182 - mae: 0.4182
Epoch 34/80
354/354             37s 104ms/step -
loss: 0.4165 - mae: 0.4165
Epoch 35/80
354/354             36s 102ms/step -
loss: 0.4116 - mae: 0.4116
Epoch 36/80
354/354             36s 102ms/step -
loss: 0.4129 - mae: 0.4129
Epoch 37/80
354/354             37s 104ms/step -
loss: 0.4070 - mae: 0.4070
Epoch 38/80
354/354             36s 102ms/step -
loss: 0.4053 - mae: 0.4053
Epoch 39/80
354/354             36s 102ms/step -
loss: 0.4039 - mae: 0.4039
Epoch 40/80
354/354             36s 102ms/step -
loss: 0.4015 - mae: 0.4015
Epoch 41/80
354/354             36s 103ms/step -
loss: 0.3994 - mae: 0.3994
```

```
Epoch 42/80
354/354             36s 102ms/step -
loss: 0.3970 - mae: 0.3970
Epoch 43/80
354/354             36s 102ms/step -
loss: 0.3939 - mae: 0.3939
Epoch 44/80
354/354             36s 102ms/step -
loss: 0.3925 - mae: 0.3925
Epoch 45/80
354/354             36s 103ms/step -
loss: 0.3910 - mae: 0.3910
Epoch 46/80
354/354             36s 101ms/step -
loss: 0.3903 - mae: 0.3903
Epoch 47/80
354/354             37s 103ms/step -
loss: 0.3875 - mae: 0.3875
Epoch 48/80
354/354             37s 103ms/step -
loss: 0.3861 - mae: 0.3861
Epoch 49/80
354/354             37s 105ms/step -
loss: 0.3848 - mae: 0.3848
Epoch 50/80
354/354             36s 101ms/step -
loss: 0.3836 - mae: 0.3836
Epoch 51/80
354/354             37s 104ms/step -
loss: 0.3813 - mae: 0.3813
Epoch 52/80
354/354             36s 102ms/step -
loss: 0.3805 - mae: 0.3805
Epoch 53/80
354/354             36s 102ms/step -
loss: 0.3796 - mae: 0.3796
Epoch 54/80
354/354             36s 102ms/step -
loss: 0.3776 - mae: 0.3776
Epoch 55/80
354/354             36s 102ms/step -
loss: 0.3775 - mae: 0.3775
Epoch 56/80
354/354             37s 104ms/step -
loss: 0.3760 - mae: 0.3760
Epoch 57/80
354/354             37s 103ms/step -
loss: 0.3750 - mae: 0.3750
```

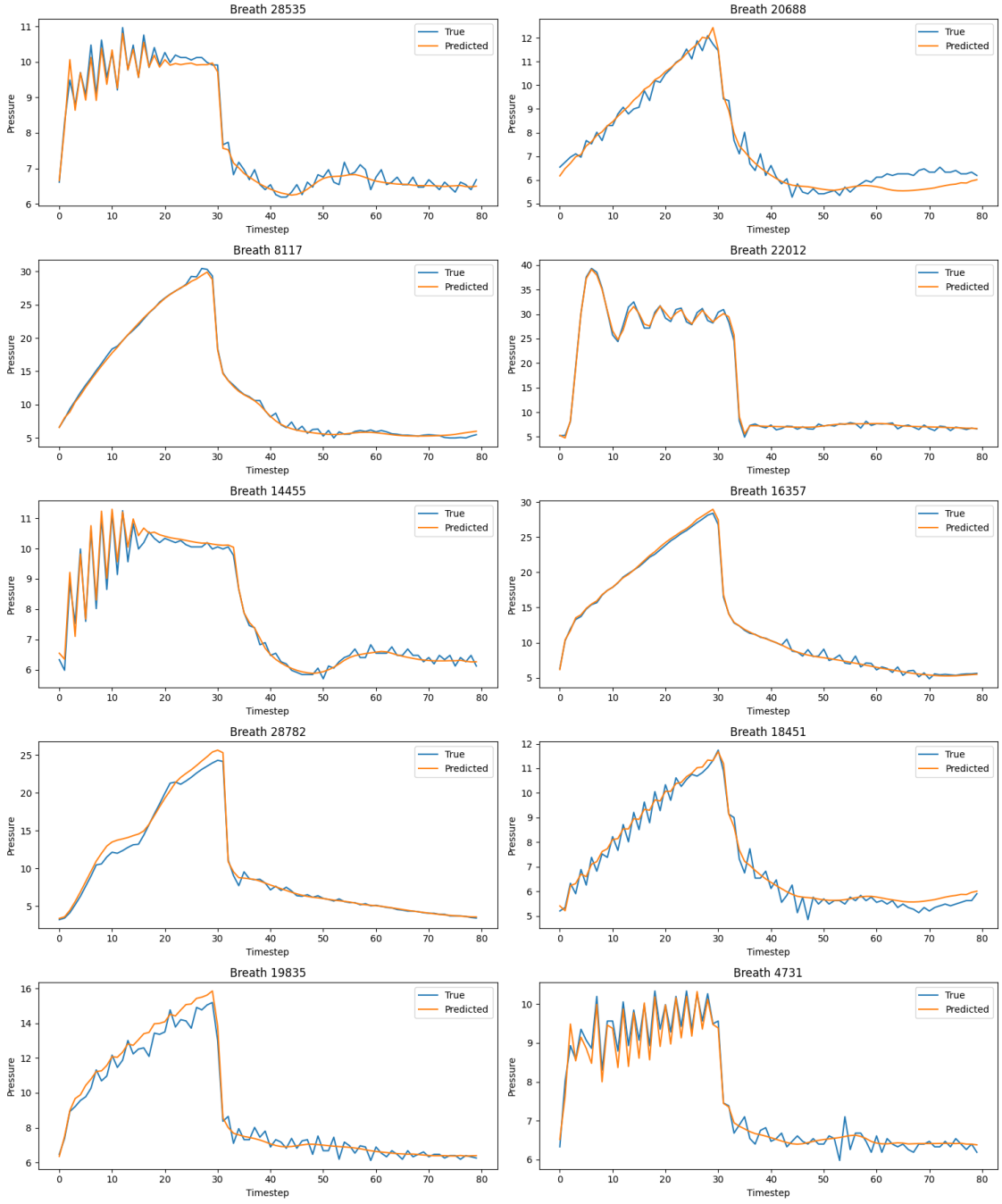
```
Epoch 58/80
354/354             37s 104ms/step -
loss: 0.3740 - mae: 0.3740
Epoch 59/80
354/354             37s 104ms/step -
loss: 0.3735 - mae: 0.3735
Epoch 60/80
354/354             36s 102ms/step -
loss: 0.3729 - mae: 0.3729
Epoch 61/80
354/354             36s 103ms/step -
loss: 0.3724 - mae: 0.3724
Epoch 62/80
354/354             36s 102ms/step -
loss: 0.3718 - mae: 0.3718
Epoch 63/80
354/354             37s 105ms/step -
loss: 0.3715 - mae: 0.3715
Epoch 64/80
354/354             37s 103ms/step -
loss: 0.3711 - mae: 0.3711
Epoch 65/80
354/354             36s 102ms/step -
loss: 0.3707 - mae: 0.3707
Epoch 66/80
354/354             36s 101ms/step -
loss: 0.3706 - mae: 0.3706
Epoch 67/80
354/354             36s 102ms/step -
loss: 0.3705 - mae: 0.3705
Epoch 68/80
354/354             37s 103ms/step -
loss: 0.3705 - mae: 0.3705
Epoch 69/80
354/354             37s 103ms/step -
loss: 0.3705 - mae: 0.3705
Epoch 70/80
354/354             36s 103ms/step -
loss: 0.3704 - mae: 0.3704
Epoch 71/80
354/354             37s 103ms/step -
loss: 0.3704 - mae: 0.3704
Epoch 72/80
354/354             37s 103ms/step -
loss: 0.3703 - mae: 0.3703
Epoch 73/80
354/354             37s 104ms/step -
loss: 0.3703 - mae: 0.3703
```

```
Epoch 74/80
354/354             36s 101ms/step -
loss: 0.3702 - mae: 0.3702
Epoch 75/80
354/354             37s 104ms/step -
loss: 0.3701 - mae: 0.3701
Epoch 76/80
354/354             37s 103ms/step -
loss: 0.3701 - mae: 0.3701
Epoch 77/80
354/354             37s 104ms/step -
loss: 0.3700 - mae: 0.3700
Epoch 78/80
354/354             37s 103ms/step -
loss: 0.3700 - mae: 0.3700
Epoch 79/80
354/354             40s 101ms/step -
loss: 0.3700 - mae: 0.3700
Epoch 80/80
354/354             36s 101ms/step -
loss: 0.3699 - mae: 0.3699
```

26 Plots

[18]:
```python
# PREDICTIONS on validation set
# ------------------------------------------------
print('='*25,'PLOT BREATH SAMPLES ON TEST','='*25)
y_pred = model.predict(test)
# ------------------------------------------------
# Plots of 10 single breath
# ------------------------------------------------
num_samples = 10
indices = np.random.choice(len(y_test), num_samples, replace=False)

plt.figure(figsize=(15, 18))

for i, idx in enumerate(indices):
    plt.subplot(5, 2, i + 1)
    plt.plot(y_test[idx].flatten())
    plt.plot(y_pred[idx].flatten())
    plt.title(f"Breath {idx}")
    plt.xlabel("Timestep")
    plt.ylabel("Pressure")
    plt.legend(["True", "Predicted"])
plt.tight_layout()
plt.show()
```

```
========================= PLOT BREATH SAMPLES ON TEST =========================
944/944                   29s 29ms/step
```



27 SHAP

[19]:
```
# ------------------------------
# Initializes the JavaScript visualization environment used by the SHAP library
# to display interactive plots in Jupyter notebooks.
# ------------------------------
shap.initjs()
```

```
<IPython.core.display.HTML object>
```

[20]:
```
# ------------------------------
# Extracting time-series dimensions
# ------------------------------
n_timesteps = train.shape[1]
n_features = train.shape[2]

# ------------------------------
# Flattening the time-series
# ------------------------------
train_flat = train.reshape(train.shape[0], -1)
test_flat = test.reshape(test.shape[0], -1)

# ------------------------------
# Define a prediction wrapper
# ------------------------------
def predict_fn(x):
    x = x.reshape(x.shape[0], n_timesteps, n_features)
    return model.predict(x)

print(train_flat.shape)
```

```
(37725, 2800)
```

[21]:
```
# ------------------------------
# Selects the first 10 samples from the flattened training set.
# ------------------------------
background = train_flat[:10]

# ------------------------------
# Creating the Kernel SHAP explainer
# ------------------------------
explainer = shap.KernelExplainer(predict_fn, background)

# ------------------------------
# Computing SHAP values
# ------------------------------
shap_values = explainer.shap_values(test_flat[:5])
```

```
print(shap_values.shape)
```

```
1/1             0s 76ms/step

  0%|          | 0/5 [00:00<?, ?it/s]

1/1             0s 70ms/step
1632/1632            46s 28ms/step

 20%|          | 1/5 [01:22<05:28, 82.00s/it]

1/1             0s 59ms/step
1732/1732            46s 27ms/step

 40%|         | 2/5 [02:39<03:57, 79.28s/it]

1/1             0s 61ms/step
1632/1632            41s 25ms/step

 60%|        | 3/5 [03:44<02:25, 72.67s/it]

1/1             0s 61ms/step
1632/1632            41s 25ms/step

 80%|       | 4/5 [04:48<01:09, 69.25s/it]

1/1             0s 63ms/step
1632/1632            41s 25ms/step

100%|      | 5/5 [05:51<00:00, 70.34s/it]

(5, 2800, 80)
```

[22]:
```
# ----------------------------
# Convert to array
# ----------------------------
shap_values_array = np.array(shap_values)  # (5, 2800, 80)

# ----------------------------
# Reshape input features back to (timesteps, features)
# ----------------------------
shap_values_array = shap_values_array.reshape(
    test_flat[:5].shape[0],  # samples
    n_timesteps,             # input timesteps
    n_features,              # features
    n_timesteps              # output timesteps
)
# (samples, input_timesteps, features, output_timesteps)
# i.e., (5, 80, 35, 80)

# ----------------------------
# Take the diagonal (own timestep features)
```

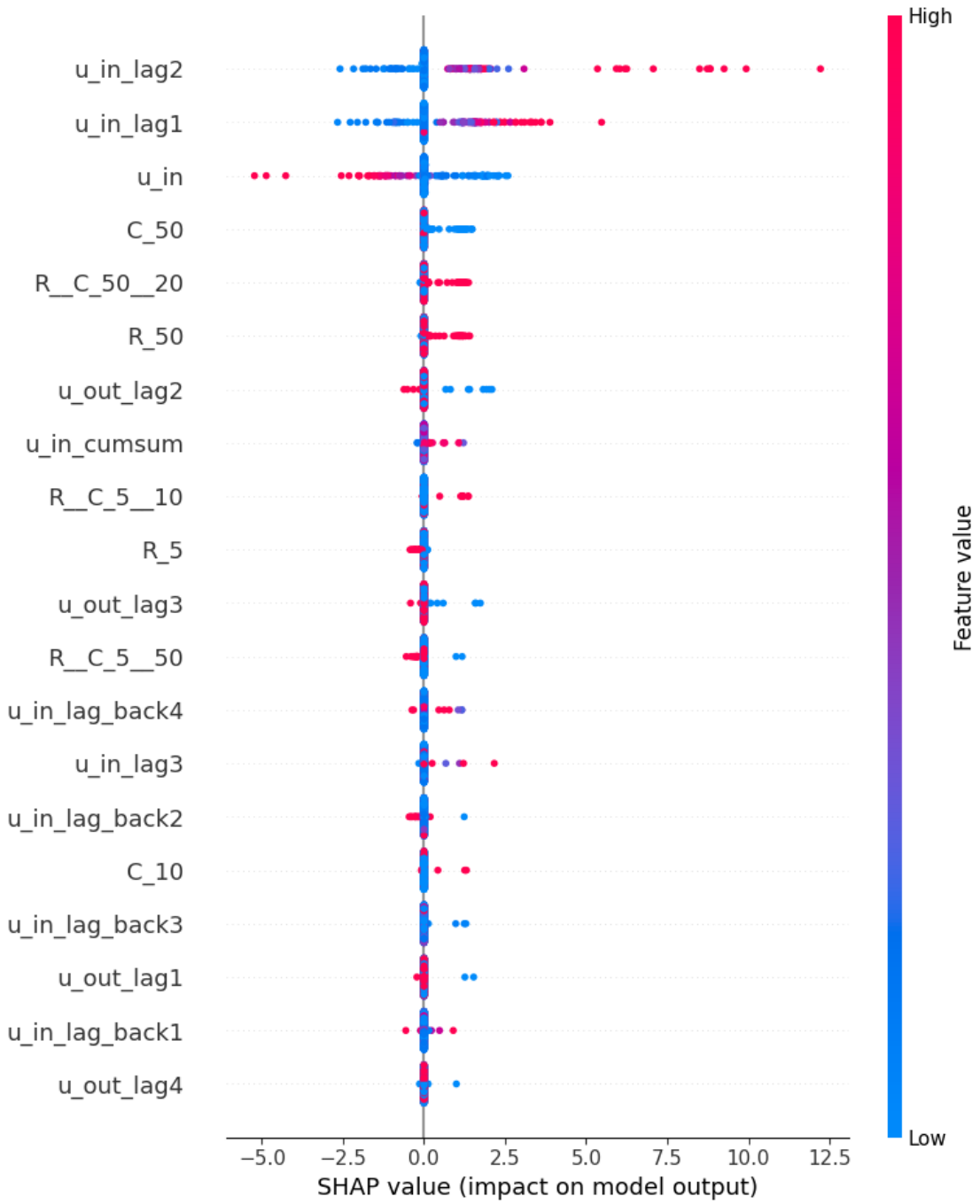
```
# ----------------------------
shap_diag = shap_values_array[np.arange(5)[:, None], np.arange(n_timesteps), :,␣
 ↪np.arange(n_timesteps)]
# shape: (samples, timesteps, features) -> (5, 80, 35)

print("SHAP diagonal shape:", shap_diag.shape)
# SHAP diagonal shape: (5, 80, 35)
```

```
SHAP diagonal shape: (5, 80, 35)
```

27.1 Shap Plots

What This Plot Represents
- Y-axis: features used by the model
- X-axis: SHAP value = impact on prediction
- Each dot: one sample from the dataset
- Distance from 0: strength of the feature's influence

[23]:
```
# flatten for summary plot: (samples*timesteps, features)
shap_flat = shap_diag.reshape(-1, n_features)
X_flat = test[:5].reshape(-1, n_features)

shap.summary_plot(
    shap_flat,
    X_flat,
    feature_names=features
)
```



Interpretation:

- SHAP > 0 $\rightarrow$ pushes prediction higher
- SHAP < 0 $\rightarrow$ pushes prediction lower
- More spread $\rightarrow$ more important feature

Top contributors appear to be:

- u_in_lag1
- u_in_lag2
- u_in

This tells us your LSTM heavily depends on recent past values.
This is exactly what we expect in time-series dynamics.

Medium Importance Features
These still influence predictions but less strongly:
- C_50 - R___C_50___20
- R_50 - R___C_5___10

These are system configuration parameters, meaning the model uses them but dynamics dominate.

One Important Limitation of This Plot

This SHAP plot collapses the time dimension.
SHAP summarizes it only per feature, not per time step.

A more informative plot would be:
feature × timestep importance
which shows **when** each feature matters.

28 SHAP Temporal Importance Heatmap

28.1 Plot heatmap

This plot is a temporal SHAP heatmap, which shows how each feature influences the prediction at each timestep of our sequence.

X-axis : Timestep (0–79) Each column represents a time position in the sequence.
Our model uses 80 timesteps.

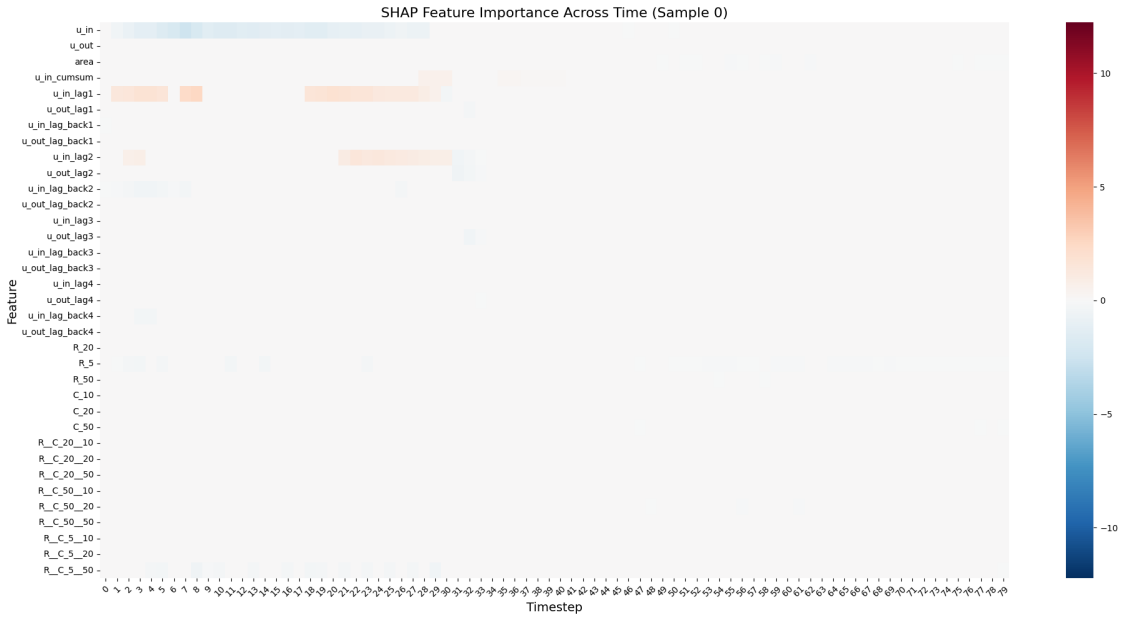
Y-axis : Features - Examples:
- u_in
- u_out
- lag features (u_in_lag1, u_out_lag2, etc.)
- system parameters (R, C)
- engineered features (area, u_in_cumsum)

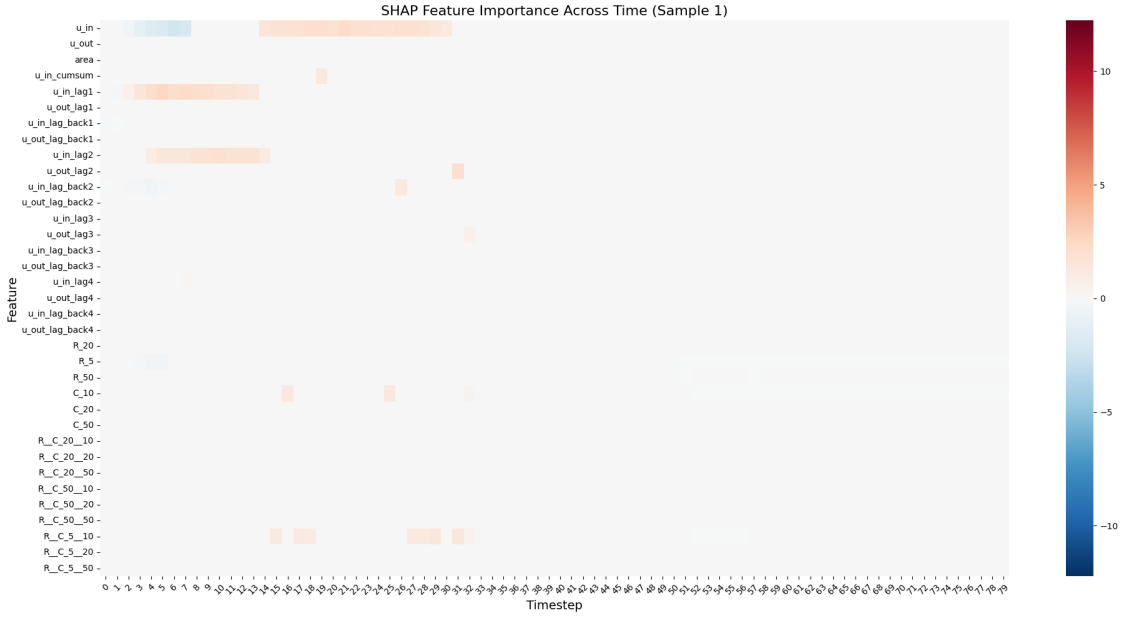
Color Meanings:

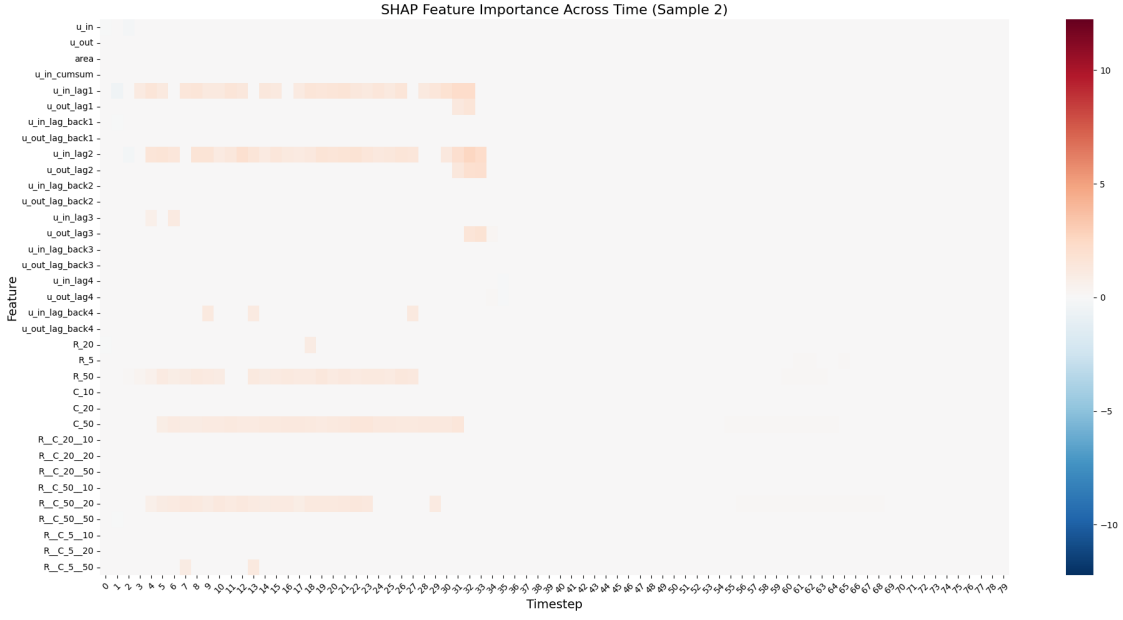
Color	Meaning
Red	Feature **increases prediction**
Blue	Feature **decreases prediction**
White	Feature **little or no effect**

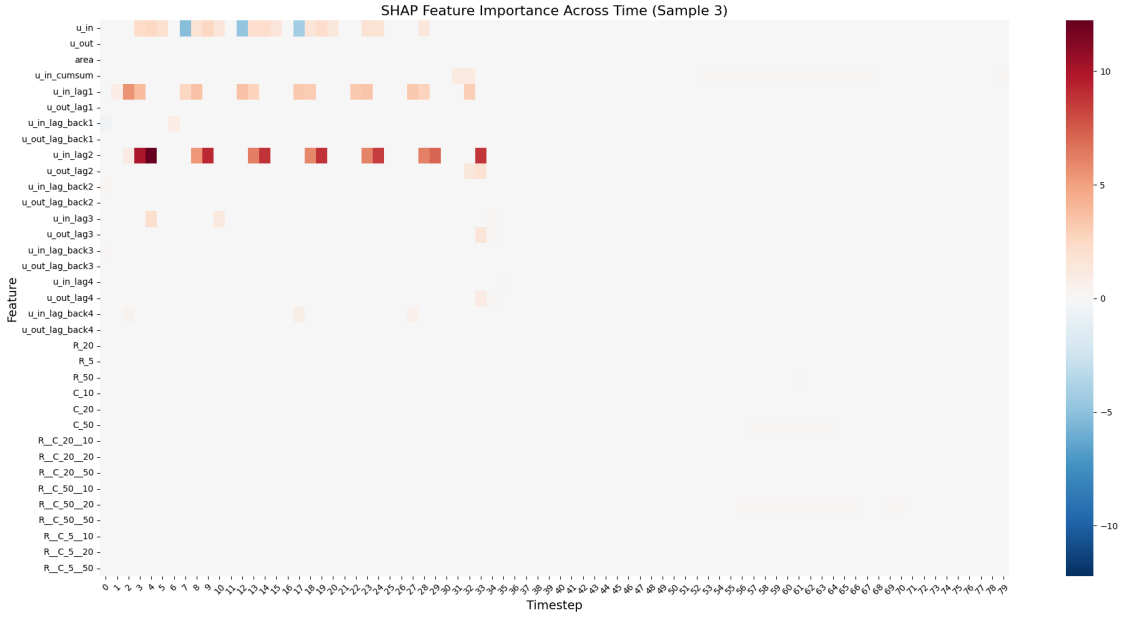
[24]:
```
for instance in range(shap_diag.shape[0]):
  plt.figure(figsize=(20, 10))  # width x height in inches
  sns.heatmap(
```

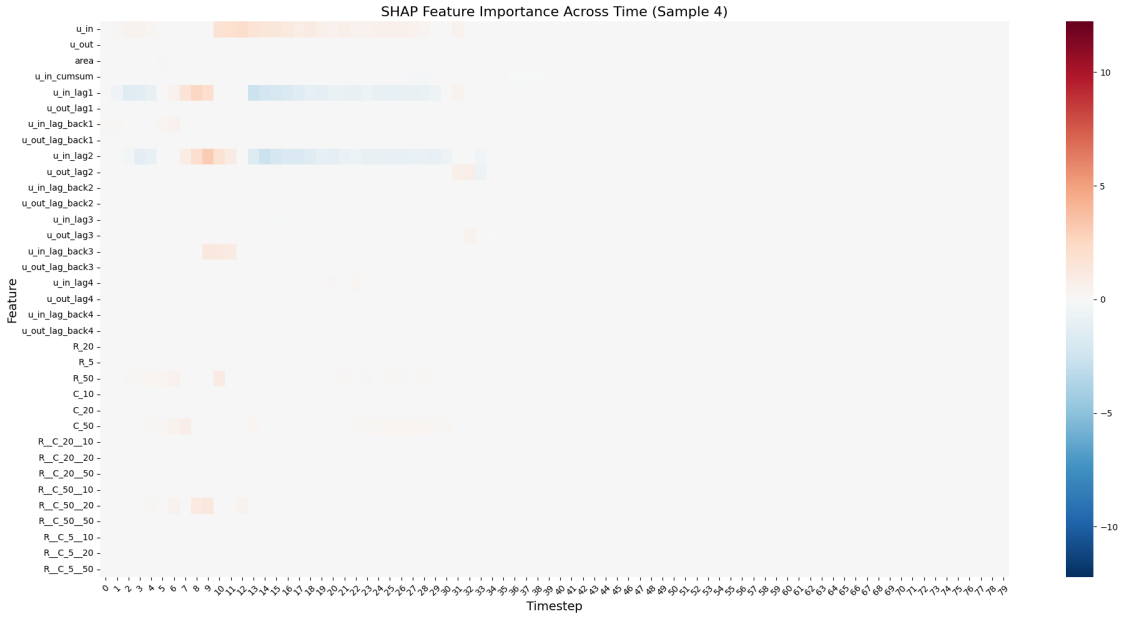
```
    shap_diag[instance].T,
    xticklabels=range(n_timesteps),
    yticklabels=features,
    cmap="RdBu_r",
    center=0,
    vmax = np.abs(shap_diag).max(),
    vmin = - np.abs(shap_diag).max(),
)

plt.xlabel("Timestep", fontsize=14)
plt.ylabel("Feature", fontsize=14)
plt.title(f"SHAP Feature Importance Across Time (Sample {instance})",␣
↪fontsize=16)
plt.xticks(rotation=45)  # rotate x-axis labels for readability
plt.yticks(rotation=0)   # keep y-axis labels horizontal
plt.tight_layout()       # adjust layout to prevent cut-offs
plt.show()
```




SHAP Feature Importance Across Time (Sample 1)
Feature
u_in
u_out
area
u_in_cumsum
u_in_lag1
u_out_lag1
u_in_lag_back1
u_out_lag_back1
u_in_lag2
u_out_lag2
u_in_lag_back2
u_out_lag_back2
u_in_lag3
u_out_lag3
u_in_lag_back3
u_out_lag_back3
u_in_lag4
u_out_lag4
u_in_lag_back4
u_out_lag_back4
R_20
R_5
R_50
C_10
C_20
C_50
R__C_20__10
R__C_20__20
R__C_20__50
R__C_50__10
R__C_50__20
R__C_50__50
R__C_5__10
R__C_5__20
R__C_5__50
Timestep
10
5
0
−5
−10


SHAP Feature Importance Across Time (Sample 2)
Feature
u_in
u_out
area
u_in_cumsum
u_in_lag1
u_out_lag1
u_in_lag_back1
u_out_lag_back1
u_in_lag2
u_out_lag2
u_in_lag_back2
u_out_lag_back2
u_in_lag3
u_out_lag3
u_in_lag_back3
u_out_lag_back3
u_in_lag4
u_out_lag4
u_in_lag_back4
u_out_lag_back4
R_20
R_5
R_50
C_10
C_20
C_50
R__C_20__10
R__C_20__20
R__C_20__50
R__C_50__10
R__C_50__20
R__C_50__50
R__C_5__10
R__C_5__20
R__C_5__50
Timestep
10
5
0
−5
−10





29 Important Patterns in our Plot

Important timesteps appear to be the initial ones: 2 - 33

Lag features are dominant: - u_in_lag1 - u_in_lag2

Late timesteps matter less.

We can notice how most of the right side of the plots are empty. That means:
t = 40–79
has almost no influence on predictions.

Caveat: SHAP plot shows local explanation, not global importance. To see global feature-time importance we should run SHAP across all samples which a task very consuming time, and almost imposible with the KernelExplainer.